

Data Encoding (Serialization) Formats — Revision Notes

Source: *Designing Data-Intensive Applications* (Martin Kleppmann), Chapter 4, "Encoding and Evolution", up to "The Merits of Schemas" (pp. 111–128). Covers: JSON / XML / CSV, MessagePack, **Apache Thrift**, **Protocol Buffers**, **Apache Avro**.

1. What is encoding (serialization)?

Programs keep data in **two different forms**:

	In memory	On disk / over the network
Shape	Objects, structs, lists, hash maps, trees	A self-contained sequence of bytes
Optimized for	Fast CPU access (uses pointers)	Storing or sending (pointers mean nothing to another process)

```
in-memory object —encode / serialize / marshal→ bytes → file / network
in-memory object ←decode / deserialize / parse / unmarshal— bytes
```

- **Encoding = serialization = marshallng. Decoding = deserialization = parsing = unmarshalling.**
- ⚠ DDIA says "encoding" because "**serialization**" also means something completely different in **transactions** (serializable isolation).
- Encoding has **nothing to do with encryption**.

2. Why it matters: schemas change over time

Old and new code, and old and new data, **live side by side**:

- **Rolling upgrades** on servers: the new version goes out to a few nodes at a time.
- **Client apps**: users may not update for a long time.

So formats must stay compatible **in both directions**:

	Meaning	Difficulty
Backward compatibility	New code reads old data	Usually easy: you know the old format
Forward compatibility	Old code reads new data	Harder: old code must ignore things it doesn't know

The running example used for every format below:

```
{ "userName": "Martin", "favoriteNumber": 1337, "interests": ["daydreaming", "hacking"] }
```

3. Language-specific formats (avoid them)

Java `java.io.Serializable` , Ruby `Marshal` , Python `pickle` , Kryo (Java).

-  Convenient: save and restore objects with almost no code.
-  **Tied to one language**, so other languages can't read the data easily.
-  **Security risk**: decoding can create **arbitrary classes**, which can let an attacker **run remote code**.
-  **Versioning is an afterthought**: poor forward and backward compatibility.
-  **Inefficient**: Java serialization is known for being slow and bloated.

➡ Use them only for **very short-lived** purposes.

4. JSON, XML and CSV (text formats)

Language-independent, widely supported and human-readable, but with subtle problems:

Problem	Detail
Number ambiguity	XML and CSV can't tell a number from a string of digits. JSON can't tell integers from floats and has no precision. Integers $> 2^{53}$ lose precision in JavaScript, so Twitter sends tweet IDs twice, as a number and as a string
No binary strings	Binary data must be Base64 -encoded, which is hacky and +33% size
Optional, complex schemas	XML Schema and JSON Schema are powerful but hard. Many JSON tools skip schemas and hard-code the decoding logic
CSV has no schema	The application must define the columns. Escaping commas and newlines is vague and often implemented wrongly

- Still **good enough** for **data exchange between organizations**, where agreeing on *any* format is the hard part.

Binary JSON variants (MessagePack, BSON, BSON, UBJSON, Smile ...)

- More compact, and some add types (int vs float, binary strings).
- But with **no schema**, they must still **include every field name** in the data.
- Example: **MessagePack = 66 bytes** vs **81 bytes** for JSON without whitespace. That's a small saving for losing human readability.

```
MessagePack: 83 | a8 "userName" | a6 "Martin" | ae "favoriteNumber" | cd 05 39 | a9 "interests" | 92
...
      |      |
      |      └ a = string, 8 = length 8
      └ 8 = object (map), 3 = three fields
```

5. Thrift and Protocol Buffers

- **Protocol Buffers (protobuf)**: from **Google**. **Thrift**: from **Facebook**. Both open-sourced in **2007–08**.
- Both **require a schema**, written in an **IDL** (interface definition language).

- A **code generator** turns the schema into classes in many languages, which you use to encode and decode.

```
// Thrift IDL                                // Protocol Buffers
struct Person {                               message Person {
  1: required string      userName,           required string user_name      = 1;
  2: optional i64         favoriteNumber,     optional int64  favorite_number = 2;
  3: optional list<string> interests         repeated string interests     = 3;
}                                              }
```

The numbers **1, 2, 3** are **field tags**, and they are the key idea.

The big trick: field tags instead of field names

The encoded bytes contain **no field names** (`userName` ...). Each field is written as **tag number + type + value**. Tags are short **aliases** for the fields.

Thrift BinaryProtocol: 59 bytes

```
0b | 00 01 | 00 00 00 06 | M a r t i n          ← type=string, tag=1, length=6, value
0a | 00 02 | 00 00 00 00 00 00 05 39           ← type=i64,    tag=2, 1337 in 8 bytes
0f | 00 03 | 0b | 00 00 00 02 |                ← type=list,   tag=3, items=string,
count=2                                         count=2
      00 00 00 0b d a y d r e a m i n g | 00 00 00 07 h a c k i n g
00                                             ← end of struct
```

Thrift CompactProtocol: 34 bytes

Same information, packed tighter:

- **Field tag + type in a single byte** (it stores the **difference** from the previous tag).
- **Variable-length integers**: 1337 takes **2 bytes** instead of 8.

```
18 | 06 | M a r t i n          ← tag delta 1, type string | length 6
16 | f2 14                    ← tag delta 1, type i64    | 1337 as a zigzag varint
19 | 28 | 0b daydreaming | 07 hacking ← tag delta 1, type list | 2 items of type string
00                             ← end of struct
```

Protocol Buffers: 33 bytes

Very similar to CompactProtocol (protobuf has **only one** binary format):

```
0a | 06 | M a r t i n          ← (tag 1 << 3) | wire type 2 (length-delimited), length 6
10 | b9 0a                    ← (tag 2 << 3) | wire type 0 (varint), 1337 as a varint
1a | 0b | d a y d r e a m i n g ← tag 3, length 11
1a | 07 | h a c k i n g        ← tag 3 AGAIN (repeated = same tag several times)
```

How variable-length integers (varints) work

Each byte holds **7 bits of the number**. The **top bit** says "more bytes follow".

```
1337 = 101 0011 1001 (binary)
split into 7-bit groups, lowest first: 0111001 | 0001010
add the "more" bit: 1 0111001 = b9 | 0 0001010 = 0a → b9 0a (2 bytes)
```

- Small numbers take fewer bytes: **-64 to 63 → 1 byte**, **-8192 to 8191 → 2 bytes** (with zigzag encoding for negatives, used by Thrift CompactProtocol and Avro).

required VS **optional**

These **don't change the encoding at all**. **required** only adds a **runtime check** that fails if the field isn't set, which helps catch bugs.

6. Schema evolution with field tags (Thrift & protobuf)

An encoded record is just its **fields joined together**, each identified by its **tag**. Unset fields are **left out**.

Change	Allowed?	Why
Rename a field	✓ Yes	The data never contains names
Change a field's tag	✗ Never	All existing data would become invalid
Add a field (new tag)	✓ If optional or with a default	Forward: old code skips unknown tags (the type annotation says how many bytes to skip). Backward: new code reading old data won't find it, so it can't be required
Remove a field	✓ Only if optional	Never reuse its tag number , because old data with that tag may still exist
Change a datatype	⚠ Risky	e.g. int32 → int64: new code reads old data fine, but old code truncates big values

Lists: protobuf **repeated** vs **Thrift** **list<>**

- **Protobuf** has no list type, only a **repeated** marker: the **same tag appears several times**. You can safely change **optional** (one value) → **repeated** (many values). New code sees a list of 0 or 1 items, and old code sees **only the last item**.
- **Thrift** has a real **list<T>** type. It can't evolve from single to multi-valued like that, but it **supports nested lists**.

7. Apache Avro

- Started in **2009** as part of **Hadoop**, because Thrift didn't fit Hadoop's needs.
- Uses a schema in one of two languages: **Avro IDL** (for humans) or **JSON** (for machines).
- **No tag numbers** in the schema.

```
// Avro IDL
record Person {
  string          userName;
  union { null, long } favoriteNumber = null;
  array<string>    interests;
}
```

```
{
  "type": "record", "name": "Person",
  "fields": [
    {"name": "userName",      "type": "string"},
    {"name": "favoriteNumber", "type": ["null", "long"], "default": null},
    {"name": "interests",     "type": {"type": "array", "items": "string"}}
  ]
}
```

Avro encoding: 32 bytes, the smallest

The bytes contain **only the values joined together: no tags, no field names, no types**.

```
0c M a r t i n          ← userName: length 6 (zigzag 12 = 0x0c), then "Martin"
02 f2 14                ← favoriteNumber: union branch 1 (long), then 1337
04 16 daydreaming 0e hacking 00 ← interests: block of 2 items, each length-prefixed, then 00 = end of array
```

- To decode, walk the fields **in schema order**, using the schema to know each type.
- So the reader must know **exactly** which schema the writer used. A mismatch would decode garbage.

The writer's schema and the reader's schema

- **Writer's schema:** the schema the app used to **encode** the data.
- **Reader's schema:** the schema the app **expects** when decoding (often compiled into the code).
- **They don't have to be identical, only compatible.** The Avro library compares them **side by side** and translates:

Writer's schema		Reader's schema	
-----		-----	
userName	→	userName	(matched by NAME, order can differ)
favoriteNumber	→	favoriteNumber	
interests	→	(not in reader)	→ IGNORED
(not in writer)		photoURL	→ filled with the reader's DEFAULT value

Avro schema evolution rules

- **Forward compatible** = new writer schema, old reader schema. **Backward compatible** = old writer schema, new reader schema.

-  You may **only add or remove fields that have a default value**.
 - Adding a field **without** a default breaks **backward** compatibility (new readers can't fill it in for old data).
 - Removing a field **without** a default breaks **forward** compatibility (old readers can't fill it in for new data).
- **Null is not a default for everything**. To allow null, use a **union**: `union { null, long } field`. Null can be the default only if it's a branch of the union (and it must be the **first** branch).
- So Avro has **no optional / required**. It uses **unions + defaults** instead.
- **Changing a type** is OK if Avro can convert it.
- **Renaming a field**: the reader schema can list **aliases**. This is backward compatible but **not forward** compatible. Adding a union branch is the same.

How does the reader know the writer's schema?

Sending the full schema with every record would be bigger than the data itself. It depends on the situation:

Situation	Solution
Big file with millions of records (Hadoop)	Put the writer's schema once at the start of the file (an object container file)
Database with records written at different times	Store a schema version number with each record and keep a table of schema versions (e.g. LinkedIn's Espresso)
Network connection	Agree on the schema version when the connection opens , then use it for the whole connection (Avro RPC)

A **schema registry** (a database of schema versions) is useful anyway: it documents the schemas and lets you **check compatibility before deploying**. The version can be a counter or a **hash of the schema**.

Why "no tag numbers" matters: dynamically generated schemas

- Example: dumping a **relational database** to files. **Generate an Avro schema automatically** from the table definitions (each table → record, each column → field, **column name** → **field name**).
- If the database schema changes, just **generate a new schema** and keep exporting. Readers match fields **by name**, so it keeps working.
- With Thrift/protobuf, someone would have to **assign field tags by hand** and never reuse old ones. Dynamic schemas weren't a design goal there.

Code generation is optional

- **Thrift/protobuf rely on code generation**. That's great for **statically typed languages** (Java, C++, C#): efficient structures, type checking and IDE autocomplete.
- In **dynamic languages** (Python, JavaScript, Ruby), generated code adds little.
- **Avro works without code generation**. An object container file **describes itself** (it embeds the writer's schema), so you can open it like a JSON file. That's handy with tools like **Apache Pig**.

8. The merits of schemas

- Thrift, protobuf and Avro schemas are **much simpler** than XML Schema or JSON Schema, which is why so many languages support them.
- The idea is old: **ASN.1** (1984) used tag numbers too, and its DER encoding is still used for **SSL/X.509 certificates**. But it's complex and badly documented.
- Databases also use **their own binary network protocols**, decoded by drivers (ODBC/JDBC).

Why schema-based binary encodings are good:

1. **Much more compact** than binary JSON, because field names are left out.
2. The **schema is documentation** that's **always up to date**, since you need it to decode.
3. A **schema database** lets you **check forward and backward compatibility before deploying**.
4. **Code generation** gives **compile-time type checking** in static languages.

➡ You get the **flexibility of schemaless JSON** with **better guarantees and better tools**.

9. Comparison table

	JSON	MessagePack	Thrift (Compact)	Protocol Buffers	Avro
Format	Text	Binary	Binary	Binary	Binary
Schema	Optional	None	Required (IDL)	Required (.proto)	Required (IDL / JSON)
Field names in data?	✅ Yes	✅ Yes	❌ Tag numbers	❌ Tag numbers	❌ Nothing , values only
Size of example	81 B	66 B	34 B (Binary: 59 B)	33 B	32 B
Fields matched by	Name	Name	Tag	Tag	Name (writer vs reader schema)
Evolution rule	Ad hoc	Ad hoc	New fields optional or defaulted, never reuse tags	Same as Thrift	Add or remove only fields with defaults
Nulls / optional	null	nil	optional	optional	union with null
Code generation	No	No	Yes	Yes	Optional
Dynamic schemas	—	—	Hard (tags by hand)	Hard (tags by hand)	✅ Easy
Origin	Web / JavaScript	—	Facebook	Google	Hadoop
Typical use	Public APIs, web	Caches, compact JSON	RPC services	gRPC* , internal services	Kafka* + schema registry, Hadoop files

* gRPC and the Kafka schema registry aren't in this part of the book, but they're the most common real-world uses today.

10. Conclusion

Encoding (serialization) turns in-memory objects into bytes for storage or the network. Because old and new code coexist during rolling upgrades, formats need **backward compatibility** (new code reads old data) and **forward compatibility** (old code reads new data). **Language-specific formats** (pickle, Java serialization) are insecure and lock you into one language. **JSON/XML/CSV** are universal but verbose and vague about numbers and binary data. **Thrift and Protocol Buffers** use a schema with **numbered field tags**, so the data carries tags instead of names (33–34 bytes vs 81). New fields must be optional or have a default, and **tags can never change or be reused**. **Avro** stores **only values** (32 bytes) and resolves differences between the **writer's schema and the reader's schema** by **field name**. You can only add or remove fields **with defaults**. It handles **dynamically generated schemas** and works without code generation, and the writer's schema travels in the file header, in a version registry, or in the connection handshake. Schema-based binary formats give **compact data, always-correct documentation and compatibility checks**.

Interview one-liners

- **Serialization:** "Converting in-memory objects to bytes for storage or the network, and back again. The key concerns are size, speed and schema evolution."
- **Protobuf/Thrift:** "Schema-based binary formats that write numbered field tags instead of names. Old code skips unknown tags, which gives forward compatibility. New fields must be optional, and tags are never reused."
- **Avro:** "No tags. The data is just values, decoded by resolving the writer's schema against the reader's schema by field name. Fields need defaults to evolve. It's great for Hadoop and Kafka with a schema registry."
- **Backward vs forward:** "Backward compatibility: new code reads old data. Forward compatibility: old code reads new data."

Self-check questions

1. What are encoding and decoding? Why does DDIA avoid the word "serialization"?
2. Define backward and forward compatibility. Which is harder, and why?
3. Why are language-specific formats like pickle or Java serialization a bad idea?
4. What problems do JSON, XML and CSV have with numbers and binary data?
5. Why is MessagePack only slightly smaller than JSON?
6. How do field tags make Thrift and protobuf compact? Why can a tag never change?
7. How does a varint encode 1337 in 2 bytes?
8. Which schema changes are safe in protobuf? Why must new fields be optional?
9. How does protobuf's `repeated` differ from Thrift's `list<>`?
10. How does Avro decode data that contains no tags or types?
11. Explain the writer's schema vs the reader's schema and how Avro resolves differences.
12. What are Avro's evolution rules? How does Avro handle null?
13. Name three ways an Avro reader can find the writer's schema.
14. Why is Avro better for dynamically generated schemas?
15. List four advantages of schema-based binary encodings.